Replit Development Prompt 11b: Portal Link Management & Communication

Prerequisites

- Prompt 11a completed with secure external document portal functionality
- Portal token system operational with document submission capabilities
- Target company and contact management systems working
- Email system configured for automated communications

What This Prompt Builds

Create comprehensive portal link management and communication system that allows internal users to generate, send, and manage secure portal access links. This includes automated email templates, portal access monitoring, link renewal capabilities, and communication tracking between internal teams and external contacts.

System Context Reminder

- **Portal Link Management**: Generate and manage secure access links for target company contacts
- **Automated Communications**: Professional email templates and automated sending
- **Access Monitoring**: Track portal usage, document submissions, and communication
- **Link Lifecycle**: Handle expiration, renewal, and revocation of portal access

What You Need to Build

1. Portal Link Management System

Create comprehensive portal link management with these capabilities:

Link Generation and Distribution:

- Generate secure portal links for specific contacts and deals
- Customizable email templates with professional branding
- Bulk link generation for multiple contacts
- Automated email sending with delivery tracking

Link Lifecycle Management:

- Monitor link expiration and usage
- Automatic renewal notifications and processes
- Manual link extension and revocation
- Link usage analytics and reporting

Communication Management:

- Track all portal-related communications
- Email template library with customization
- Automated follow-up sequences
- Communication history and audit trail

2. Database Schema for Portal Management

```sql
-- Create portal link campaigns table
CREATE TABLE portal_link_campaigns (
  id SERIAL PRIMARY KEY,
  user_company_id INTEGER REFERENCES user_companies(id),
  target_company_id INTEGER REFERENCES target_companies(id),
  created_by INTEGER REFERENCES users(id),

  -- Campaign Details
  campaign_name VARCHAR(255) NOT NULL,
  campaign_description TEXT,
  campaign_type VARCHAR(50) DEFAULT 'document_collection', --
document_collection, information_request, follow_up

  -- Email Configuration
  email_template_id INTEGER,
  custom_subject VARCHAR(255),
  custom_message TEXT,
  sender_name VARCHAR(255),
  sender_email VARCHAR(255),

  -- Campaign Status
  campaign_status VARCHAR(50) DEFAULT 'draft', -- draft, active, paused,
completed, cancelled
  scheduled_send_time TIMESTAMP,
  sent_at TIMESTAMP,

  -- Tracking
  total_recipients INTEGER DEFAULT 0,
  emails_sent INTEGER DEFAULT 0,
  emails_delivered INTEGER DEFAULT 0,
```

```
  emails_opened INTEGER DEFAULT 0,
  links_accessed INTEGER DEFAULT 0,
  documents_submitted INTEGER DEFAULT 0,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Create email templates table
CREATE TABLE portal_email_templates (
  id SERIAL PRIMARY KEY,
  user_company_id INTEGER REFERENCES user_companies(id),
  created_by INTEGER REFERENCES users(id),

  -- Template Details
  template_name VARCHAR(255) NOT NULL,
  template_description TEXT,
  template_category VARCHAR(50) DEFAULT 'portal_invitation', --
portal_invitation, reminder, follow_up, renewal

  -- Email Content
  email_subject VARCHAR(255) NOT NULL,
  email_body_html TEXT NOT NULL,
  email_body_text TEXT,

  -- Template Variables
  available_variables JSONB, -- List of variables that can be used in template

  -- Template Settings
  is_default BOOLEAN DEFAULT FALSE,
  is_active BOOLEAN DEFAULT TRUE,

  -- Usage Tracking
  usage_count INTEGER DEFAULT 0,
  last_used TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Create portal communications log table
CREATE TABLE portal_communications_log (
  id SERIAL PRIMARY KEY,
  portal_token_id INTEGER REFERENCES portal_access_tokens(id),
  campaign_id INTEGER REFERENCES portal_link_campaigns(id),
  target_contact_id INTEGER REFERENCES target_contacts(id),
```

```
  -- Communication Details
  communication_type VARCHAR(50) NOT NULL, -- email_sent, email_opened,
email_clicked, portal_accessed, document_submitted
  communication_subject VARCHAR(255),
  communication_content TEXT,

  -- Email Tracking
  email_message_id VARCHAR(255), -- External email service message ID
  email_status VARCHAR(50), -- sent, delivered, bounced, opened, clicked

  -- Timing
  sent_at TIMESTAMP,
  delivered_at TIMESTAMP,
  opened_at TIMESTAMP,
  clicked_at TIMESTAMP,

  -- Technical Details
  recipient_email VARCHAR(255),
  sender_email VARCHAR(255),
  ip_address INET,
  user_agent TEXT,

  -- Response Tracking
  response_received BOOLEAN DEFAULT FALSE,
  response_content TEXT,
  response_timestamp TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW()
);

-- Create portal analytics table
CREATE TABLE portal_analytics (
  id SERIAL PRIMARY KEY,
  user_company_id INTEGER REFERENCES user_companies(id),
  target_company_id INTEGER REFERENCES target_companies(id),
  portal_token_id INTEGER REFERENCES portal_access_tokens(id),

  -- Date Tracking
  analytics_date DATE DEFAULT CURRENT_DATE,

  -- Access Metrics
  unique_visitors INTEGER DEFAULT 0,
  total_page_views INTEGER DEFAULT 0,
  session_duration_avg INTEGER DEFAULT 0, -- seconds
  bounce_rate DECIMAL(5,2) DEFAULT 0.00,

  -- Document Metrics
```

```
  documents_uploaded INTEGER DEFAULT 0,
  documents_approved INTEGER DEFAULT 0,
  documents_rejected INTEGER DEFAULT 0,
  total_file_size BIGINT DEFAULT 0,

  -- Communication Metrics
  messages_sent INTEGER DEFAULT 0,
  messages_received INTEGER DEFAULT 0,
  response_time_avg INTEGER DEFAULT 0, -- hours

  -- Completion Metrics
  requirements_completed INTEGER DEFAULT 0,
  requirements_total INTEGER DEFAULT 0,
  completion_percentage DECIMAL(5,2) DEFAULT 0.00,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(user_company_id, target_company_id, portal_token_id, analytics_date)
);

-- Create portal link sharing table
CREATE TABLE portal_link_shares (
  id SERIAL PRIMARY KEY,
  portal_token_id INTEGER REFERENCES portal_access_tokens(id),
  shared_by INTEGER REFERENCES users(id),

  -- Sharing Details
  sharing_method VARCHAR(50) NOT NULL, -- email, sms, direct_link, qr_code
  recipient_email VARCHAR(255),
  recipient_phone VARCHAR(50),

  -- Sharing Configuration
  share_message TEXT,
  share_expiry TIMESTAMP,
  max_uses INTEGER DEFAULT 1,
  current_uses INTEGER DEFAULT 0,

  -- Tracking
  is_active BOOLEAN DEFAULT TRUE,
  first_accessed TIMESTAMP,
  last_accessed TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW()
);

-- Insert default email templates
```

```
INSERT INTO portal_email_templates (user_company_id, template_name,
template_description, template_category, email_subject, email_body_html,
available_variables) VALUES
(1, 'Portal Invitation - Initial', 'Initial invitation to access document
submission portal', 'portal_invitation',
'Document Submission Portal Access - {{target_company_name}}',
'<html><body>
<h2>Document Submission Portal Access</h2>
<p>Dear {{contact_first_name}} {{contact_last_name}},</p>
<p>We are pleased to provide you with secure access to our document submission
portal for the potential acquisition of {{target_company_name}}.</p>
<p><strong>Portal Access:</strong><br>
<a href="{{portal_url}}" style="background-color: #007bff; color: white;
padding: 12px 24px; text-decoration: none; border-radius: 4px; display:
inline-block;">Access Document Portal</a></p>
<p><strong>What you need to do:</strong></p>
<ul>
<li>Click the link above to access the secure portal</li>
<li>Review the list of required documents</li>
<li>Upload documents as they become available</li>
<li>Use the messaging system for any questions</li>
</ul>
<p>This portal access will remain active until {{expiry_date}}. If you need an
extension, please let us know.</p>
<p>If you have any questions or technical issues, please contact us at
{{sender_email}}.</p>
<p>Best regards,<br>{{sender_name}}<br>{{acquiring_company_name}}</p>
</body></html>',
'["contact_first_name", "contact_last_name", "target_company_name",
"portal_url", "expiry_date", "sender_name", "sender_email",
"acquiring_company_name"]'::jsonb),

(1, 'Portal Reminder - Gentle', 'Gentle reminder for portal access and document
submission', 'reminder',
'Reminder: Document Submission Portal - {{target_company_name}}',
'<html><body>
<h2>Friendly Reminder: Document Portal Access</h2>
<p>Dear {{contact_first_name}},</p>
<p>We hope this message finds you well. This is a friendly reminder about the
document submission portal for {{target_company_name}}.</p>
<p><strong>Current Status:</strong></p>
<ul>
<li>Documents submitted: {{documents_submitted}} of {{total_requirements}}</li>
<li>Completion: {{completion_percentage}}%</li>
<li>Portal expires: {{expiry_date}}</li>
</ul>
```

```
<p><a href="{{portal_url}}" style="background-color: #28a745; color: white;
padding: 10px 20px; text-decoration: none; border-radius: 4px;">Continue
Submission</a></p>
<p>We understand that gathering documents takes time. If you need any
assistance or have questions about specific requirements, please don\'t
hesitate to reach out.</p>
<p>Thank you for your cooperation.</p>
<p>Best regards,<br>{{sender_name}}<br>{{acquiring_company_name}}</p>
</body></html>',
'["contact_first_name", "target_company_name", "documents_submitted",
"total_requirements", "completion_percentage", "expiry_date", "portal_url",
"sender_name", "acquiring_company_name"]'::jsonb),

(1, 'Portal Expiry Warning', 'Warning about upcoming portal expiry', 'renewal',
'Action Required: Portal Access Expiring Soon - {{target_company_name}}',
'<html><body>
<h2>Portal Access Expiring Soon</h2>
<p>Dear {{contact_first_name}},</p>
<p><strong>Important:</strong> Your document submission portal access will
expire in {{days_until_expiry}} days on {{expiry_date}}.</p>
<p><strong>Current Progress:</strong></p>
<ul>
<li>{{completion_percentage}}% complete</li>
<li>{{outstanding_documents}} documents still needed</li>
</ul>
<p>To ensure continuity of the process, please:</p>
<ol>
<li><a href="{{portal_url}}">Access the portal</a> and submit any remaining
documents</li>
<li>Contact us if you need more time - we can extend access</li>
<li>Let us know if you\'re experiencing any difficulties</li>
</ol>
<p>We\'re here to help make this process as smooth as possible.</p>
<p>Best regards,<br>{{sender_name}}<br>{{acquiring_company_name}}</p>
</body></html>',
'["contact_first_name", "target_company_name", "days_until_expiry",
"expiry_date", "completion_percentage", "outstanding_documents", "portal_url",
"sender_name", "acquiring_company_name"]'::jsonb);

-- Create indexes for performance
CREATE INDEX idx_portal_campaigns_company ON
portal_link_campaigns(user_company_id, campaign_status);
CREATE INDEX idx_portal_campaigns_target ON
portal_link_campaigns(target_company_id, campaign_status);
CREATE INDEX idx_email_templates_company ON
portal_email_templates(user_company_id, is_active);
```

```
CREATE INDEX idx_communications_log_token ON
portal_communications_log(portal_token_id, communication_type);
CREATE INDEX idx_communications_log_contact ON
portal_communications_log(target_contact_id, sent_at);
CREATE INDEX idx_portal_analytics_date ON portal_analytics(analytics_date,
user_company_id);
CREATE INDEX idx_portal_shares_token ON portal_link_shares(portal_token_id,
is_active);
```

3. API Endpoints for Portal Management

JavaScript

```
// Portal link management (internal users)
POST /api/portal-management/campaigns            // Create portal link campaign
GET /api/portal-management/campaigns             // Get campaigns for company
PUT /api/portal-management/campaigns/:id         // Update campaign
POST /api/portal-management/campaigns/:id/send   // Send campaign emails
DELETE /api/portal-management/campaigns/:id      // Delete campaign

// Email template management
GET /api/portal-management/templates             // Get email templates
POST /api/portal-management/templates            // Create email template
PUT /api/portal-management/templates/:id         // Update email template
DELETE /api/portal-management/templates/:id      // Delete email template

// Portal analytics and monitoring
GET /api/portal-management/analytics             // Get portal analytics
GET /api/portal-management/communications        // Get communication logs
GET /api/portal-management/tokens/status         // Get token status overview
POST /api/portal-management/tokens/:id/extend    // Extend token expiration

// Portal sharing and distribution
POST /api/portal-management/share                // Share portal link
GET /api/portal-management/shares                // Get sharing history
PUT /api/portal-management/shares/:id            // Update sharing settings

// Communication tracking
GET /api/portal-management/messages              // Get portal messages
POST /api/portal-management/messages/reply       // Reply to portal message
PUT /api/portal-management/messages/:id/status   // Update message status
```

4. Portal Management Implementation

JavaScript

```
const portalLinkManager = {
  // Create and send portal link campaign
  createPortalCampaign: async (campaignData, createdBy) => {
    // Verify permissions
    const user = await getUserWithPermissions(createdBy);
    if (!user.can_manage_users && !user.can_upload_documents) {
      throw new Error('Insufficient permissions to create portal campaigns');
    }

    // Create campaign record
    const campaignQuery = `
      INSERT INTO portal_link_campaigns (
        user_company_id, target_company_id, created_by, campaign_name,
        campaign_description, campaign_type, email_template_id,
        custom_subject, custom_message, sender_name, sender_email,
        scheduled_send_time
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12)
      RETURNING *
    `;

    const campaignValues = [
      user.user_company_id,
      campaignData.target_company_id,
      createdBy,
      campaignData.campaign_name,
      campaignData.campaign_description || '',
      campaignData.campaign_type || 'document_collection',
      campaignData.email_template_id,
      campaignData.custom_subject || '',
      campaignData.custom_message || '',
      campaignData.sender_name || `${user.first_name} ${user.last_name}`,
      campaignData.sender_email || user.email,
      campaignData.scheduled_send_time || null
    ];

    const result = await db.query(campaignQuery, campaignValues);
    const campaign = result.rows[0];

    // If immediate send requested, process now
    if (campaignData.send_immediately) {
      await sendPortalCampaign(campaign.id);
    }

    return campaign;
```

```
  },

  // Send portal campaign emails
  sendPortalCampaign: async (campaignId) => {
    // Get campaign details
    const campaignQuery = `
      SELECT
        plc.*,
        tc.company_name as target_company_name,
        uc.company_name as acquiring_company_name
      FROM portal_link_campaigns plc
      JOIN target_companies tc ON plc.target_company_id = tc.id
      JOIN user_companies uc ON plc.user_company_id = uc.id
      WHERE plc.id = $1
    `;

    const campaignResult = await db.query(campaignQuery, [campaignId]);
    if (campaignResult.rows.length === 0) {
      throw new Error('Campaign not found');
    }

    const campaign = campaignResult.rows[0];

    // Get email template
    const template = await getEmailTemplate(campaign.email_template_id);
    if (!template) {
      throw new Error('Email template not found');
    }

    // Get target contacts
    const contactsQuery = `
      SELECT * FROM target_contacts
      WHERE target_company_id = $1 AND is_active = true
    `;
    const contacts = await db.query(contactsQuery,
[campaign.target_company_id]);

    let emailsSent = 0;
    let emailsDelivered = 0;

    // Send email to each contact
    for (const contact of contacts.rows) {
      try {
        // Generate or get existing portal token for this contact
        let portalToken = await
getOrCreatePortalToken(campaign.target_company_id, contact.id,
campaign.created_by);
```

```
        // Prepare email variables
        const emailVariables = {
          contact_first_name: contact.first_name,
          contact_last_name: contact.last_name,
          target_company_name: campaign.target_company_name,
          acquiring_company_name: campaign.acquiring_company_name,
          portal_url: `${process.env.APP_BASE_URL ||
'https://your-app.replit.app'}/portal/${portalToken.access_token}`,
          expiry_date: new Date(portalToken.expires_at).toLocaleDateString(),
          sender_name: campaign.sender_name,
          sender_email: campaign.sender_email
        };

        // Render email content
        const emailSubject = renderTemplate(campaign.custom_subject ||
template.email_subject, emailVariables);
        const emailBody = renderTemplate(template.email_body_html,
emailVariables);

        // Send email
        const emailResult = await sendEmail({
          to: contact.email,
          from: campaign.sender_email,
          subject: emailSubject,
          html: emailBody,
          campaign_id: campaignId,
          portal_token_id: portalToken.id
        });

        if (emailResult.success) {
          emailsSent++;
          if (emailResult.delivered) emailsDelivered++;

          // Log communication
          await logPortalCommunication(portalToken.id, campaignId, contact.id,
'email_sent', emailSubject, emailBody, contact.email, campaign.sender_email,
emailResult.message_id);
        }

      } catch (error) {
        console.error(`Failed to send email to ${contact.email}:`, error);

        // Log failed communication
        await logPortalCommunication(null, campaignId, contact.id,
'email_failed', 'Email Send Failed', error.message, contact.email,
campaign.sender_email);
```

```
      }
    }

    // Update campaign statistics
    await db.query(`
      UPDATE portal_link_campaigns
      SET
        campaign_status = 'active',
        sent_at = NOW(),
        total_recipients = $2,
        emails_sent = $3,
        emails_delivered = $4
      WHERE id = $1
    `, [campaignId, contacts.rows.length, emailsSent, emailsDelivered]);

    return {
      campaign_id: campaignId,
      total_recipients: contacts.rows.length,
      emails_sent: emailsSent,
      emails_delivered: emailsDelivered
    };
  },

  // Get or create portal token for contact
  getOrCreatePortalToken: async (targetCompanyId, contactId, createdBy) => {
    // Check for existing active token
    const existingTokenQuery = `
      SELECT * FROM portal_access_tokens
      WHERE target_company_id = $1
      AND target_contact_id = $2
      AND is_active = true
      AND expires_at > NOW()
      ORDER BY created_at DESC
      LIMIT 1
    `;

    const existingResult = await db.query(existingTokenQuery, [targetCompanyId,
contactId]);

    if (existingResult.rows.length > 0) {
      return existingResult.rows[0];
    }

    // Create new token
    return await portalTokenManager.generatePortalToken(targetCompanyId,
contactId, createdBy);
  },
```

```
  // Monitor portal analytics
  updatePortalAnalytics: async (portalTokenId, analyticsData) => {
    const token = await db.query('SELECT * FROM portal_access_tokens WHERE id =
$1', [portalTokenId]);
    if (token.rows.length === 0) return;

    const tokenData = token.rows[0];

    const analyticsQuery = `
      INSERT INTO portal_analytics (
        user_company_id, target_company_id, portal_token_id,
        unique_visitors, total_page_views, session_duration_avg,
        documents_uploaded, messages_sent, messages_received
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
      ON CONFLICT (user_company_id, target_company_id, portal_token_id,
analytics_date)
      DO UPDATE SET
        unique_visitors = portal_analytics.unique_visitors +
EXCLUDED.unique_visitors,
        total_page_views = portal_analytics.total_page_views +
EXCLUDED.total_page_views,
        session_duration_avg = (portal_analytics.session_duration_avg +
EXCLUDED.session_duration_avg) / 2,
        documents_uploaded = portal_analytics.documents_uploaded +
EXCLUDED.documents_uploaded,
        messages_sent = portal_analytics.messages_sent +
EXCLUDED.messages_sent,
        messages_received = portal_analytics.messages_received +
EXCLUDED.messages_received,
        updated_at = NOW()
    `;

    const values = [
      tokenData.user_company_id,
      tokenData.target_company_id,
      portalTokenId,
      analyticsData.unique_visitors || 0,
      analyticsData.total_page_views || 0,
      analyticsData.session_duration_avg || 0,
      analyticsData.documents_uploaded || 0,
      analyticsData.messages_sent || 0,
      analyticsData.messages_received || 0
    ];

    await db.query(analyticsQuery, values);
  },
```

```
  // Get portal analytics dashboard
  getPortalAnalytics: async (userCompanyId, dateRange = 30) => {
    const analyticsQuery = `
      SELECT
        DATE_TRUNC('day', analytics_date) as date,
        SUM(unique_visitors) as unique_visitors,
        SUM(total_page_views) as total_page_views,
        AVG(session_duration_avg) as avg_session_duration,
        SUM(documents_uploaded) as documents_uploaded,
        SUM(messages_sent) as messages_sent,
        SUM(messages_received) as messages_received,
        AVG(completion_percentage) as avg_completion_percentage
      FROM portal_analytics
      WHERE user_company_id = $1
      AND analytics_date >= CURRENT_DATE - INTERVAL '$2 days'
      GROUP BY DATE_TRUNC('day', analytics_date)
      ORDER BY date DESC
    `;

    const result = await db.query(analyticsQuery, [userCompanyId, dateRange]);

    // Get summary statistics
    const summaryQuery = `
      SELECT
        COUNT(DISTINCT portal_token_id) as active_portals,
        SUM(unique_visitors) as total_visitors,
        SUM(documents_uploaded) as total_documents,
        AVG(completion_percentage) as avg_completion,
        COUNT(CASE WHEN completion_percentage = 100 THEN 1 END) as
completed_portals
      FROM portal_analytics
      WHERE user_company_id = $1
      AND analytics_date >= CURRENT_DATE - INTERVAL '$2 days'
    `;

    const summaryResult = await db.query(summaryQuery, [userCompanyId,
dateRange]);

    return {
      daily_analytics: result.rows,
      summary: summaryResult.rows[0]
    };
  }
};

const emailTemplateManager = {
```

```
// Create custom email template
createEmailTemplate: async (templateData, createdBy) => {
  const user = await getUserWithPermissions(createdBy);

  const templateQuery = `
    INSERT INTO portal_email_templates (
      user_company_id, created_by, template_name, template_description,
      template_category, email_subject, email_body_html, email_body_text,
      available_variables
    ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
    RETURNING *
  `;

  const values = [
    user.user_company_id,
    createdBy,
    templateData.template_name,
    templateData.template_description || '',
    templateData.template_category || 'portal_invitation',
    templateData.email_subject,
    templateData.email_body_html,
    templateData.email_body_text || '',
    JSON.stringify(templateData.available_variables || [])
  ];

  const result = await db.query(templateQuery, values);
  return result.rows[0];
},

// Get email templates for company
getEmailTemplates: async (userCompanyId, category = null) => {
  let templatesQuery = `
    SELECT * FROM portal_email_templates
    WHERE user_company_id = $1 AND is_active = true
  `;
  const queryParams = [userCompanyId];

  if (category) {
    templatesQuery += ' AND template_category = $2';
    queryParams.push(category);
  }

  templatesQuery += ' ORDER BY is_default DESC, template_name ASC';

  const result = await db.query(templatesQuery, queryParams);
  return result.rows;
},
```

```
  // Render email template with variables
  renderTemplate: (template, variables) => {
    let rendered = template;

    // Replace all {{variable}} placeholders
    Object.keys(variables).forEach(key => {
      const regex = new RegExp(`{{${key}}}`, 'g');
      rendered = rendered.replace(regex, variables[key] || '');
    });

    return rendered;
  }
};

const portalCommunicationManager = {
  // Log portal communication
  logPortalCommunication: async (portalTokenId, campaignId, contactId,
communicationType, subject, content, recipientEmail, senderEmail, messageId =
null) => {
    const logQuery = `
      INSERT INTO portal_communications_log (
        portal_token_id, campaign_id, target_contact_id, communication_type,
        communication_subject, communication_content, email_message_id,
        recipient_email, sender_email, sent_at
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, NOW())
      RETURNING *
    `;

    const values = [
      portalTokenId,
      campaignId,
      contactId,
      communicationType,
      subject,
      content,
      messageId,
      recipientEmail,
      senderEmail
    ];

    const result = await db.query(logQuery, values);
    return result.rows[0];
  },

  // Get communication history
```

```
  getCommunicationHistory: async (userCompanyId, targetCompanyId = null, limit
= 100) => {
    let historyQuery = `
      SELECT
        pcl.*,
        tc.first_name,
        tc.last_name,
        tc.email,
        tcomp.company_name
      FROM portal_communications_log pcl
      JOIN target_contacts tc ON pcl.target_contact_id = tc.id
      JOIN target_companies tcomp ON tc.target_company_id = tcomp.id
      WHERE tcomp.user_company_id = $1
    `;

    const queryParams = [userCompanyId];

    if (targetCompanyId) {
      historyQuery += ' AND tcomp.id = $2';
      queryParams.push(targetCompanyId);
    }

    historyQuery += ' ORDER BY pcl.created_at DESC LIMIT $' +
(queryParams.length + 1);
    queryParams.push(limit);

    const result = await db.query(historyQuery, queryParams);
    return result.rows;
  },

  // Update email tracking status
  updateEmailTracking: async (messageId, trackingData) => {
    const updateQuery = `
      UPDATE portal_communications_log
      SET
        email_status = $2,
        delivered_at = $3,
        opened_at = $4,
        clicked_at = $5
      WHERE email_message_id = $1
      RETURNING *
    `;

    const values = [
      messageId,
      trackingData.status,
      trackingData.delivered_at || null,
```

```
      trackingData.opened_at || null,
      trackingData.clicked_at || null
    ];

    const result = await db.query(updateQuery, values);
    return result.rows[0];
  }
};

// Email sending function (integrate with your email service)
const sendEmail = async (emailData) => {
  try {
    // This is a placeholder - integrate with your email service (SendGrid,
Mailgun, etc.)
    console.log('Sending email:', {
      to: emailData.to,
      from: emailData.from,
      subject: emailData.subject
    });

    // Simulate email sending
    const messageId = `msg_${Date.now()}_${Math.random().toString(36).substr(2,
9)}`;

    // Log the email send attempt
    await portalCommunicationManager.logPortalCommunication(
      emailData.portal_token_id,
      emailData.campaign_id,
      null,
      'email_sent',
      emailData.subject,
      emailData.html,
      emailData.to,
      emailData.from,
      messageId
    );

    return {
      success: true,
      delivered: true,
      message_id: messageId
    };

  } catch (error) {
    console.error('Email send failed:', error);
    return {
      success: false,
```

```
      error: error.message
    };
  }
};

const getEmailTemplate = async (templateId) => {
  const result = await db.query('SELECT * FROM portal_email_templates WHERE id
= $1', [templateId]);
  return result.rows[0];
};

const renderTemplate = (template, variables) => {
  let rendered = template;

  Object.keys(variables).forEach(key => {
    const regex = new RegExp(`{{${key}}}`, 'g');
    rendered = rendered.replace(regex, variables[key] || '');
  });

  return rendered;
};
```

UI Implementation

Portal Management Dashboard

HTML

```
<div class="portal-management-dashboard">
  <div class="dashboard-header">
    <h2>Portal Link Management</h2>
    <div class="dashboard-controls">
      <button onclick="showCreateCampaignModal()" class="btn-primary">📧 Create
Campaign</button>
      <button onclick="showTemplateManager()" class="btn-secondary">📝 Manage
Templates</button>
      <button onclick="refreshAnalytics()" class="btn-secondary">📊 Refresh
Analytics</button>
    </div>
  </div>

  <!-- Analytics Overview -->
  <div class="analytics-overview">
    <h3>Portal Analytics (Last 30 Days)</h3>
```

```
    <div class="analytics-cards">
      <div class="analytics-card">
        <div class="card-value">{{analytics.summary.active_portals}}</div>
        <div class="card-label">Active Portals</div>
        <div class="card-trend">📈 +12% vs last month</div>
      </div>

      <div class="analytics-card">
        <div class="card-value">{{analytics.summary.total_visitors}}</div>
        <div class="card-label">Total Visitors</div>
        <div class="card-trend">👥 {{analytics.summary.avg_completion}}% avg
completion</div>
      </div>

      <div class="analytics-card">
        <div class="card-value">{{analytics.summary.total_documents}}</div>
        <div class="card-label">Documents Submitted</div>
        <div class="card-trend">📄 {{analytics.summary.completed_portals}}
portals completed</div>
      </div>

      <div class="analytics-card">
        <div class="card-value">{{communication_stats.response_rate}}%</div>
        <div class="card-label">Email Response Rate</div>
        <div class="card-trend">📧 {{communication_stats.avg_response_time}}h
avg response</div>
      </div>
    </div>

    <!-- Analytics Chart -->
    <div class="analytics-chart">
      <canvas id="portal-analytics-chart" width="800" height="300"></canvas>
    </div>
  </div>

  <!-- Active Campaigns -->
  <div class="campaigns-section">
    <h3>Portal Link Campaigns</h3>

    <div class="campaigns-list">
      {{#each campaigns}}
      <div class="campaign-item status-{{campaign_status}}">
        <div class="campaign-header">
          <h4>{{campaign_name}}</h4>
          <div class="campaign-meta">
            <span class="campaign-type">{{campaign_type | replace '_' ' ' |
capitalize}}</span>
```

```
            <span class="campaign-status
status-{{campaign_status}}">{{campaign_status | capitalize}}</span>
            <span class="campaign-date">{{created_at | timeAgo}}</span>
          </div>
        </div>

        <div class="campaign-content">
          <div class="campaign-description">{{campaign_description}}</div>

          <div class="campaign-stats">
            <div class="stat-item">
              <span class="stat-value">{{total_recipients}}</span>
              <span class="stat-label">Recipients</span>
            </div>
            <div class="stat-item">
              <span class="stat-value">{{emails_sent}}</span>
              <span class="stat-label">Sent</span>
            </div>
            <div class="stat-item">
              <span class="stat-value">{{emails_delivered}}</span>
              <span class="stat-label">Delivered</span>
            </div>
            <div class="stat-item">
              <span class="stat-value">{{links_accessed}}</span>
              <span class="stat-label">Accessed</span>
            </div>
            <div class="stat-item">
              <span class="stat-value">{{documents_submitted}}</span>
              <span class="stat-label">Documents</span>
            </div>
          </div>
        </div>

        <div class="campaign-actions">
          {{#if (eq campaign_status 'draft')}}
          <button onclick="sendCampaign({{id}})" class="btn-primary">📤 Send
Now</button>
          <button onclick="editCampaign({{id}})" class="btn-secondary">✏️
Edit</button>
          {{/if}}

          {{#if (eq campaign_status 'active')}}
          <button onclick="viewCampaignDetails({{id}})"
class="btn-secondary">👁️ View Details</button>
          <button onclick="pauseCampaign({{id}})" class="btn-secondary">⏸️
Pause</button>
          {{/if}}
```

```
          <button onclick="duplicateCampaign({{id}})" class="btn-secondary">📋
Duplicate</button>
          <button onclick="deleteCampaign({{id}})" class="btn-danger">🗑️
Delete</button>
        </div>
      </div>
      {{/each}}
    </div>
  </div>

  <!-- Portal Tokens Status -->
  <div class="tokens-section">
    <h3>Portal Access Tokens</h3>

    <div class="tokens-filter">
      <select id="tokens-filter" onchange="filterTokens(this.value)">
        <option value="all">All Tokens</option>
        <option value="active">Active</option>
        <option value="expiring">Expiring Soon</option>
        <option value="expired">Expired</option>
      </select>

      <input type="text" id="tokens-search" placeholder="Search by company or
contact..." onkeyup="searchTokens(this.value)">
    </div>

    <div class="tokens-list">
      {{#each portalTokens}}
      <div class="token-item {{token_status}}">
        <div class="token-info">
          <div class="token-company">
            <h5>{{target_company_name}}</h5>
            <p>{{contact_first_name}} {{contact_last_name}}
({{contact_email}})</p>
          </div>

          <div class="token-stats">
            <div class="stat-small">
              <span class="stat-value">{{access_count}}</span>
              <span class="stat-label">Accesses</span>
            </div>
            <div class="stat-small">
              <span class="stat-value">{{documents_submitted}}</span>
              <span class="stat-label">Documents</span>
            </div>
            <div class="stat-small">
```

```
              <span class="stat-value">{{completion_percentage}}%</span>
              <span class="stat-label">Complete</span>
            </div>
          </div>

          <div class="token-status">
            <span class="status-badge {{token_status}}">
              {{#if (eq token_status 'active')}}✅ Active{{/if}}
              {{#if (eq token_status 'expiring')}}⚠️ Expiring{{/if}}
              {{#if (eq token_status 'expired')}}❌ Expired{{/if}}
            </span>
            <span class="expiry-date">Expires: {{expires_at |
formatDate}}</span>
            {{#if last_accessed}}
            <span class="last-access">Last access: {{last_accessed |
timeAgo}}</span>
            {{/if}}
          </div>
        </div>

        <div class="token-actions">
          <button onclick="viewPortalActivity({{id}})" class="btn-small">📊
Activity</button>
          <button onclick="openPortalLink('{{access_token}}')"
class="btn-small">🔗 Open Portal</button>

          {{#if (eq token_status 'active')}}
          <button onclick="extendToken({{id}})" class="btn-small">⏰
Extend</button>
          {{/if}}

          {{#if (eq token_status 'expiring')}}
          <button onclick="renewToken({{id}})" class="btn-small btn-warning">🔄
Renew</button>
          {{/if}}

          {{#if (eq token_status 'expired')}}
          <button onclick="reactivateToken({{id}})" class="btn-small
btn-success">✅ Reactivate</button>
          {{/if}}

          <button onclick="revokeToken({{id}})" class="btn-small btn-danger">🚫
Revoke</button>
        </div>
      </div>
      {{/each}}
    </div>
```

```
  </div>

  <!-- Communication History -->
  <div class="communications-section">
    <h3>Recent Communications</h3>

    <div class="communications-list">
      {{#each communications}}
      <div class="communication-item type-{{communication_type}}">
        <div class="communication-icon">
          {{#if (eq communication_type 'email_sent')}}📧{{/if}}
          {{#if (eq communication_type 'email_opened')}}👁️{{/if}}
          {{#if (eq communication_type 'portal_accessed')}}🔗{{/if}}
          {{#if (eq communication_type 'document_submitted')}}📄{{/if}}
        </div>

        <div class="communication-details">
          <div class="communication-header">
            <span
class="communication-subject">{{communication_subject}}</span>
            <span class="communication-time">{{sent_at | timeAgo}}</span>
          </div>

          <div class="communication-meta">
            <span class="contact-info">{{first_name}} {{last_name}}
({{company_name}})</span>
            <span class="communication-status
status-{{email_status}}">{{email_status | capitalize}}</span>
          </div>

          {{#if communication_content}}
          <div class="communication-preview">{{communication_content | truncate
100}}</div>
          {{/if}}
        </div>

        <div class="communication-actions">
          <button onclick="viewCommunicationDetails({{id}})"
class="btn-small">👁️ Details</button>
          {{#if (eq communication_type 'email_sent')}}
          <button onclick="resendEmail({{id}})" class="btn-small">🔄
Resend</button>
          {{/if}}
        </div>
      </div>
      {{/each}}
    </div>
```

```
  </div>
</div>

<!-- Create Campaign Modal -->
<div id="create-campaign-modal" class="modal" style="display: none;">
  <div class="modal-content large">
    <div class="modal-header">
      <h3>Create Portal Link Campaign</h3>
      <button onclick="closeModal('create-campaign-modal')"
class="close-btn">×</button>
    </div>

    <form id="create-campaign-form">
      <div class="form-section">
        <h4>Campaign Details</h4>

        <div class="form-row">
          <div class="form-group">
            <label for="campaign-name">Campaign Name</label>
            <input type="text" id="campaign-name" name="campaign_name" required
                   placeholder="e.g., Initial Document Request - ABC Corp">
          </div>

          <div class="form-group">
            <label for="campaign-type">Campaign Type</label>
            <select id="campaign-type" name="campaign_type">
              <option value="document_collection">Document Collection</option>
              <option value="information_request">Information Request</option>
              <option value="follow_up">Follow-up</option>
              <option value="reminder">Reminder</option>
            </select>
          </div>
        </div>

        <div class="form-group">
          <label for="campaign-description">Description</label>
          <textarea id="campaign-description" name="campaign_description"
rows="2"
                    placeholder="Brief description of this
campaign..."></textarea>
        </div>

        <div class="form-group">
          <label for="target-company">Target Company</label>
          <select id="target-company" name="target_company_id" required>
            <option value="">Select target company...</option>
            {{#each targetCompanies}}
```

```
            <option value="{{id}}">{{company_name}} ({{industry}})</option>
            {{/each}}
          </select>
        </div>
      </div>

      <div class="form-section">
        <h4>Email Configuration</h4>

        <div class="form-row">
          <div class="form-group">
            <label for="email-template">Email Template</label>
            <select id="email-template" name="email_template_id" required
onchange="previewTemplate(this.value)">
              <option value="">Select template...</option>
              {{#each emailTemplates}}
              <option value="{{id}}">{{template_name}}
({{template_category}})</option>
              {{/each}}
            </select>
          </div>

          <div class="form-group">
            <label for="sender-name">Sender Name</label>
            <input type="text" id="sender-name" name="sender_name"
                   value="{{currentUser.first_name}}
{{currentUser.last_name}}">
          </div>
        </div>

        <div class="form-group">
          <label for="custom-subject">Custom Subject (Optional)</label>
          <input type="text" id="custom-subject" name="custom_subject"
                 placeholder="Override template subject...">
        </div>

        <div class="form-group">
          <label for="custom-message">Custom Message (Optional)</label>
          <textarea id="custom-message" name="custom_message" rows="3"
                    placeholder="Add custom message to template..."></textarea>
        </div>
      </div>

      <div class="form-section">
        <h4>Sending Options</h4>

        <div class="form-row">
```

```
          <div class="form-group">
            <label>
              <input type="checkbox" name="send_immediately" checked> Send
immediately
            </label>
          </div>

          <div class="form-group">
            <label for="scheduled-time">Or schedule for later:</label>
            <input type="datetime-local" id="scheduled-time"
name="scheduled_send_time">
          </div>
        </div>
      </div>

      <div class="template-preview" id="template-preview" style="display:
none;">
        <h4>Email Preview</h4>
        <div class="preview-content">
          <div class="preview-subject" id="preview-subject"></div>
          <div class="preview-body" id="preview-body"></div>
        </div>
      </div>

      <div class="form-actions">
        <button type="submit">Create Campaign</button>
        <button type="button"
onclick="closeModal('create-campaign-modal')">Cancel</button>
      </div>
    </form>
  </div>
</div>

<!-- Email Template Manager Modal -->
<div id="template-manager-modal" class="modal" style="display: none;">
  <div class="modal-content large">
    <div class="modal-header">
      <h3>Email Template Manager</h3>
      <button onclick="closeModal('template-manager-modal')"
class="close-btn">×</button>
    </div>

    <div class="template-manager-content">
      <div class="template-controls">
        <button onclick="createNewTemplate()" class="btn-primary">➕ New
Template</button>
```

```
        <select id="template-category-filter"
onchange="filterTemplates(this.value)">
          <option value="">All Categories</option>
          <option value="portal_invitation">Portal Invitations</option>
          <option value="reminder">Reminders</option>
          <option value="follow_up">Follow-ups</option>
          <option value="renewal">Renewals</option>
        </select>
      </div>

      <div class="templates-list">
        {{#each emailTemplates}}
        <div class="template-item">
          <div class="template-header">
            <h5>{{template_name}}</h5>
            <div class="template-meta">
              <span class="template-category">{{template_category}}</span>
              {{#if is_default}}<span
class="default-badge">Default</span>{{/if}}
              <span class="usage-count">Used {{usage_count}} times</span>
            </div>
          </div>

          <div class="template-content">
            <div class="template-subject">Subject: {{email_subject}}</div>
            <div class="template-preview">{{email_body_html | stripHtml |
truncate 150}}</div>
          </div>

          <div class="template-actions">
            <button onclick="editTemplate({{id}})" class="btn-small">✏️
Edit</button>
            <button onclick="duplicateTemplate({{id}})" class="btn-small">📋
Duplicate</button>
            <button onclick="previewTemplate({{id}})" class="btn-small">👁️
Preview</button>
            {{#unless is_default}}
            <button onclick="deleteTemplate({{id}})" class="btn-small
btn-danger">🗑️ Delete</button>
            {{/unless}}
          </div>
        </div>
        {{/each}}
      </div>
    </div>
  </div>
</div>
```

Portal Management JavaScript

JavaScript

```javascript
// Portal management functionality
document.addEventListener('DOMContentLoaded', () => {
  if (document.querySelector('.portal-management-dashboard')) {
    loadPortalAnalytics();
    loadCampaigns();
    loadPortalTokens();
    loadCommunications();

    // Auto-refresh every 5 minutes
    setInterval(refreshDashboard, 300000);
  }
});

// Campaign management
document.getElementById('create-campaign-form').addEventListener('submit',
async (e) => {
  e.preventDefault();

  const formData = new FormData(e.target);
  const campaignData = Object.fromEntries(formData.entries());

  // Convert checkbox to boolean
  campaignData.send_immediately = formData.has('send_immediately');

  try {
    const response = await fetch('/api/portal-management/campaigns', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(campaignData)
    });

    const result = await response.json();

    if (response.ok) {
      showNotification('Campaign created successfully', 'success');
      closeModal('create-campaign-modal');
      refreshCampaigns();
    } else {
      showNotification(result.error || 'Failed to create campaign', 'error');
    }

  } catch (error) {
```

```
    console.error('Campaign creation failed:', error);
    showNotification('Failed to create campaign', 'error');
  }
});

const sendCampaign = async (campaignId) => {
  if (confirm('Are you sure you want to send this campaign? This will send
emails to all contacts.')) {
    try {
      const response = await
fetch(`/api/portal-management/campaigns/${campaignId}/send`, {
        method: 'POST'
      });

      const result = await response.json();

      if (response.ok) {
        showNotification(`Campaign sent to ${result.emails_sent} recipients`,
'success');
        refreshCampaigns();
      } else {
        showNotification(result.error || 'Failed to send campaign', 'error');
      }

    } catch (error) {
      console.error('Campaign send failed:', error);
      showNotification('Failed to send campaign', 'error');
    }
  }
};

const extendToken = async (tokenId) => {
  const days = prompt('Extend token by how many days?', '30');

  if (days && !isNaN(days)) {
    try {
      const response = await
fetch(`/api/portal-management/tokens/${tokenId}/extend`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ extend_days: parseInt(days) })
      });

      if (response.ok) {
        showNotification(`Token extended by ${days} days`, 'success');
```

```
        refreshPortalTokens();
      } else {
        showNotification('Failed to extend token', 'error');
      }

    } catch (error) {
      console.error('Token extension failed:', error);
      showNotification('Failed to extend token', 'error');
    }
  }
};

const openPortalLink = (accessToken) => {
  const portalUrl = `${window.location.origin}/portal/${accessToken}`;
  window.open(portalUrl, '_blank');
};

const viewPortalActivity = async (tokenId) => {
  try {
    const response = await fetch(`/api/portal/${tokenId}/activity`);
    const activity = await response.json();

    if (response.ok) {
      showPortalActivityModal(activity);
    } else {
      showNotification('Failed to load portal activity', 'error');
    }

  } catch (error) {
    console.error('Failed to load portal activity:', error);
    showNotification('Failed to load portal activity', 'error');
  }
};

const previewTemplate = async (templateId) => {
  if (!templateId) {
    document.getElementById('template-preview').style.display = 'none';
    return;
  }

  try {
    const response = await
fetch(`/api/portal-management/templates/${templateId}`);
    const template = await response.json();

    if (response.ok) {
      // Show preview with sample variables
```

```
      const sampleVariables = {
        contact_first_name: 'John',
        contact_last_name: 'Smith',
        target_company_name: 'Sample Company Ltd',
        acquiring_company_name: 'Your Company',
        portal_url: 'https://your-app.replit.app/portal/sample-token',
        expiry_date: new Date(Date.now() + 30 * 24 * 60 * 60 *
1000).toLocaleDateString(),
        sender_name: 'Your Name',
        sender_email: 'your.email@company.com'
      };

      const previewSubject = renderTemplatePreview(template.email_subject,
sampleVariables);
      const previewBody = renderTemplatePreview(template.email_body_html,
sampleVariables);

      document.getElementById('preview-subject').textContent = previewSubject;
      document.getElementById('preview-body').innerHTML = previewBody;
      document.getElementById('template-preview').style.display = 'block';
    }

  } catch (error) {
    console.error('Template preview failed:', error);
  }
};

const renderTemplatePreview = (template, variables) => {
  let rendered = template;

  Object.keys(variables).forEach(key => {
    const regex = new RegExp(`{{${key}}}`, 'g');
    rendered = rendered.replace(regex, variables[key] || '');
  });

  return rendered;
};

const loadPortalAnalytics = async () => {
  try {
    const response = await fetch('/api/portal-management/analytics');
    const analytics = await response.json();

    if (response.ok) {
      updateAnalyticsDisplay(analytics);
      renderAnalyticsChart(analytics.daily_analytics);
    }
```

```
  } catch (error) {
    console.error('Failed to load analytics:', error);
  }
};

const renderAnalyticsChart = (dailyData) => {
  const ctx =
document.getElementById('portal-analytics-chart').getContext('2d');

  new Chart(ctx, {
    type: 'line',
    data: {
      labels: dailyData.map(d => new Date(d.date).toLocaleDateString()),
      datasets: [
        {
          label: 'Portal Visitors',
          data: dailyData.map(d => d.unique_visitors),
          borderColor: '#007bff',
          backgroundColor: 'rgba(0, 123, 255, 0.1)',
          tension: 0.4
        },
        {
          label: 'Documents Uploaded',
          data: dailyData.map(d => d.documents_uploaded),
          borderColor: '#28a745',
          backgroundColor: 'rgba(40, 167, 69, 0.1)',
          tension: 0.4
        }
      ]
    },
    options: {
      responsive: true,
      scales: {
        y: {
          beginAtZero: true
        }
      },
      plugins: {
        legend: {
          position: 'top'
        },
        title: {
          display: true,
          text: 'Portal Activity Over Time'
        }
      }
```

```
    }
  });
};

const filterTokens = (status) => {
  const tokenItems = document.querySelectorAll('.token-item');

  tokenItems.forEach(item => {
    if (status === 'all' || item.classList.contains(status)) {
      item.style.display = 'block';
    } else {
      item.style.display = 'none';
    }
  });
};

const searchTokens = (searchTerm) => {
  const tokenItems = document.querySelectorAll('.token-item');
  const term = searchTerm.toLowerCase();

  tokenItems.forEach(item => {
    const companyName = item.querySelector('.token-company
h5').textContent.toLowerCase();
    const contactName = item.querySelector('.token-company
p').textContent.toLowerCase();

    if (companyName.includes(term) || contactName.includes(term)) {
      item.style.display = 'block';
    } else {
      item.style.display = 'none';
    }
  });
};

const refreshDashboard = () => {
  loadPortalAnalytics();
  refreshCampaigns();
  refreshPortalTokens();
  refreshCommunications();
};

// Utility functions
const showPortalActivityModal = (activity) => {
  // Create and show modal with portal activity details
  const modal = document.createElement('div');
  modal.className = 'modal';
  modal.innerHTML = `
```

```
    <div class="modal-content">
      <div class="modal-header">
        <h3>Portal Activity Log</h3>
        <button onclick="this.closest('.modal').remove()"
class="close-btn">×</button>
      </div>
      <div class="activity-log">
        ${activity.map(a => `
          <div class="activity-item">
            <div class="activity-time">${new
Date(a.activity_timestamp).toLocaleString()}</div>
            <div class="activity-type">${a.activity_type.replace('_', '
').toUpperCase()}</div>
            <div class="activity-description">${a.activity_description}</div>
          </div>
        `).join('')}
      </div>
    </div>
  `;

  document.body.appendChild(modal);
  modal.style.display = 'flex';
};

// Auto-refresh functions
const refreshCampaigns = async () => {
  // Refresh campaigns list
  try {
    const response = await fetch('/api/portal-management/campaigns');
    const campaigns = await response.json();

    if (response.ok) {
      updateCampaignsDisplay(campaigns);
    }
  } catch (error) {
    console.error('Failed to refresh campaigns:', error);
  }
};

const refreshPortalTokens = async () => {
  // Refresh portal tokens list
  try {
    const response = await fetch('/api/portal-management/tokens/status');
    const tokens = await response.json();

    if (response.ok) {
      updateTokensDisplay(tokens);
```

```
    }
  } catch (error) {
    console.error('Failed to refresh tokens:', error);
  }
};

const refreshCommunications = async () => {
  // Refresh communications list
  try {
    const response = await fetch('/api/portal-management/communications');
    const communications = await response.json();

    if (response.ok) {
      updateCommunicationsDisplay(communications);
    }
  } catch (error) {
    console.error('Failed to refresh communications:', error);
  }
};
```

File Structure Updates

None

```
routes/
├── portal-management.js (new)

public/
├── js/
│   └── portal-management.js (new)
└── css/
    └── portal-management.css (new)

views/
├── portal-management/
│   ├── dashboard.ejs (new)
│   ├── campaigns.ejs (new)
│   └── templates.ejs (new)
```

Success Criteria for Prompt 11b

- [ ] Portal link campaigns can be created and managed
- [ ] Email templates system with customization and variables
- [ ] Automated email sending with delivery tracking

- [ ] Portal access monitoring and analytics
- [ ] Link expiration management and renewal
- [ ] Communication history and audit trail
- [ ] Professional email templates with branding
- [ ] Bulk operations for multiple contacts
- [ ] Real-time analytics and reporting
- [ ] Integration with existing portal system

Business Value

This prompt enables:

- **Professional Communication** - Branded email templates and automated sending
- **Efficient Management** - Bulk operations and campaign management
- **Comprehensive Tracking** - Full audit trail of all communications
- **Proactive Management** - Automated reminders and expiration warnings
- **Data-Driven Insights** - Analytics on portal usage and engagement

Integration with Existing System

This prompt integrates seamlessly with:

- **Prompt 11a**: Builds on secure portal functionality
- **Target Company Management**: Uses existing contact data
- **Team Management**: Notifications and permissions
- **Future Chase-up System**: Provides foundation for automated follow-ups

The portal link management system transforms document collection from a manual process into a professional, automated workflow that enhances client experience while providing comprehensive tracking and analytics!